BCSE301L_SOFTWARE ENGINEERING

7

Presented By,
Dr.Baiju B V
Assistant Professor,
SCOPE, VIT, Vellore

Product and Process Metrics

1. Product Metrics-Introduction:

- Product metrics are **predictor metrics** used to quantify internal attributes of a software system.

Examples :

- ✓ Size of the code (like how many lines of code it has)
- ✓ Number of methods (how many functions each class has)
- ✓ Complexity (how complicated the code is to follow)

Software characteristics that can be ***easily measured (***such as size and cyclomatic complexity), ***do not have a clear and consistent relationship with quality attributes*** such as **understandability** and **maintainability.**

Let's say we have two software programs that perform the same task.

- ***Program A has 500 lines of code, very clean and well-documented.***
- ***Program B has 300 lines of code, but it's messy and hard to read.***

Product metrics fall into **two** classes:

1. Dynamic Metrics

- These are measurements taken ***while the program is running***.
  - ➢ Collected during testing or when users are actually using the software.
  - ➢ Help us understand ***how the system behaves in real life***.
- Example
  - ✓ Number of bug reports
  - ✓ Time taken to complete a task.

2. Static metrics

- These are measurements taken ***without running the program***.
  - ➢ Collected from the code, design, or documents.
  - ➢ Help us understand ***how the system is built***.
- Example
  - ✓ Number of lines of code
  - ✓ Number of functions or classes

Static Software Product Metrics

Software metric	Description
Fan-in/ Fan-out	**Fan-In:** Means how many ***other functions are calling a function*** (let's say Function X). Many functions depend on **X** ***If X changes, it could affect many parts of the system.*** **Fan-Out :** Means how many ***functions are called by Function X***. High fan-out means X is controlling or using many other functions. ***X could be complex and hard to manage, because it needs to coordinate with many parts.***
Length of Code	This measures how big a program or a part of it is, based on the number of lines of code. • More lines of code usually means the code is more complex. • Bigger code is often more likely to have bugs.
Cyclomatic complexity	This measures ***how complicated the flow of a program*** is — like how it makes decisions, loops, or branches.
Length of Identifiers	It shows how long the names (like variables or methods) are in a program. Longer names often mean the code is easier to understand.

Static Software Product Metrics

Software metric	Description
Fog index	It tells how hard a document is to read. If the words and sentences are long, the Fog Index is high — meaning it's harder to understand. **Example**: Easy: *The dog ran fast.* Hard: *The domesticated canine accelerated rapidly.*
Depth of conditional nesting	It shows how many if-statements are inside each other. More nesting = harder to read and more chance of mistakes

Process Metrics

- Process metrics focus on the efficiency and effectiveness of testing activities.

1. **Test Case Effectiveness:** Measures how well test cases detect defects.
    1. **Formula:** (Defects Detected / Test Cases Run) x 100
    2. **Use Case:** Helps evaluate the quality of test cases and refine them for better defect detection.
2. **Cycle Time:** Tracks the time taken to complete the testing process.
    1. **Insight:** Helps teams understand the efficiency of test runs and identify delays.
    2. **Example:** If a testing cycle takes longer than expected, it may indicate inefficiencies in the test environment or setup.
3. **Defect Fixing Time:** Measures the time taken to resolve a defect from detection to closure.
    1. **Use Case:** Identifies delays in defect resolution, aiding in optimizing workflows.

- Used for assessing maintainability are:

(1) Number of bug fix requests::

- If more bugs are reported over time, it could mean new problems are being added faster than they are fixed.
- This shows the software is getting harder to maintain.

(2) Average time required for impact analysis:

- If a change affects many parts of the program and takes more time to analyze, it means the software is harder to maintain.

(3) Time to make a change:

- This is the time taken to make changes to the system and update documents after finding what needs to be changed.
- If this time keeps increasing, it may mean the software is becoming harder to maintain.

(4) Number of pending change requests::

- If this number keeps going up, it could mean the software is becoming harder to maintain.

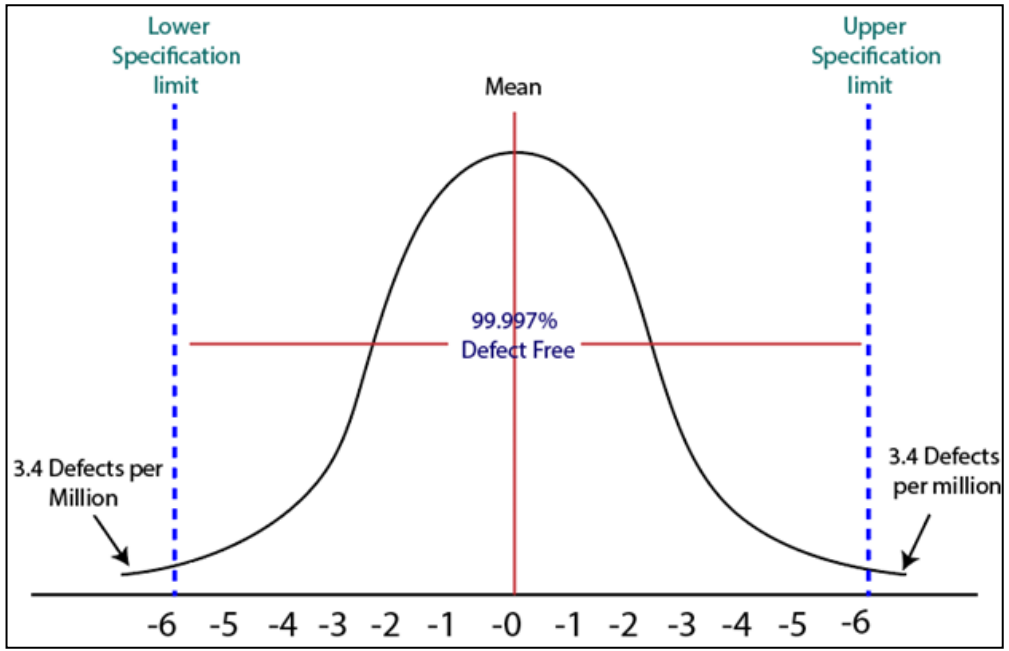
Aspect	Process Metrics (How you build it)	Product Metrics (What you build)
Focus	Development processes and workflows	Final software product and its characteristics
Purpose	Improve efficiency and effectiveness of development	Ensure the product meets quality and performance goals
Examples	Defect density, cycle time, test coverage	Defect count, Mean time between failure, usability, performance metrics
Timeframe	Measured throughout the development lifecycle	Measured primarily after the product is built or released
Impact	Long-term impact on process improvement	Immediate impact on product quality and user experience

Quality Standards Models - Six Sigma

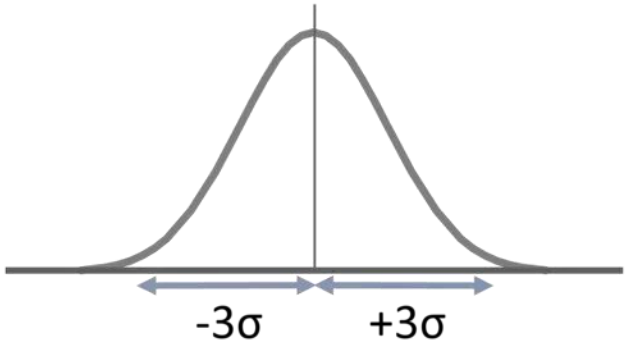
- Six Sigma is a widely used strategy in industries for quality improvement.
- It focuses on ***identifying and eliminating the causes of defects***.
- It also aims to ***reduce variations in manufacturing and business processes***.
- The maturity of a manufacturing process is measured by a sigma rating, showing the percentage of defect-free products.

- In Six Sigma, 99.99966% of the products are identical and free from defects.

In a pen factory producing 1 million pens, a Six Sigma process ensures that only 3 or 4 pens will be defective, while the remaining will be flawless.

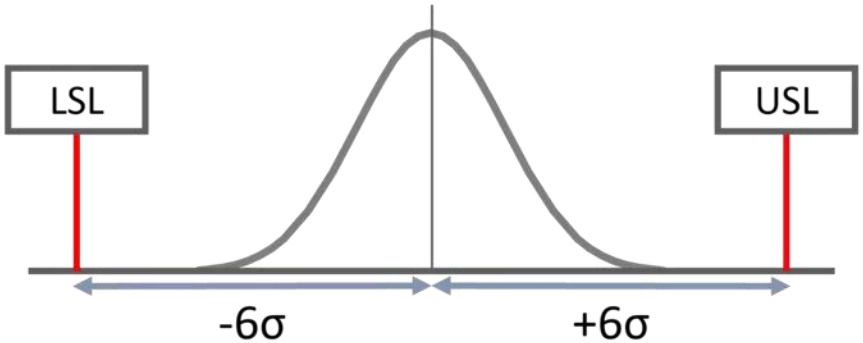


Process Variation



Customer specification

6σ = 99.99966% within LSL - USL



Characteristics of Six Sigma

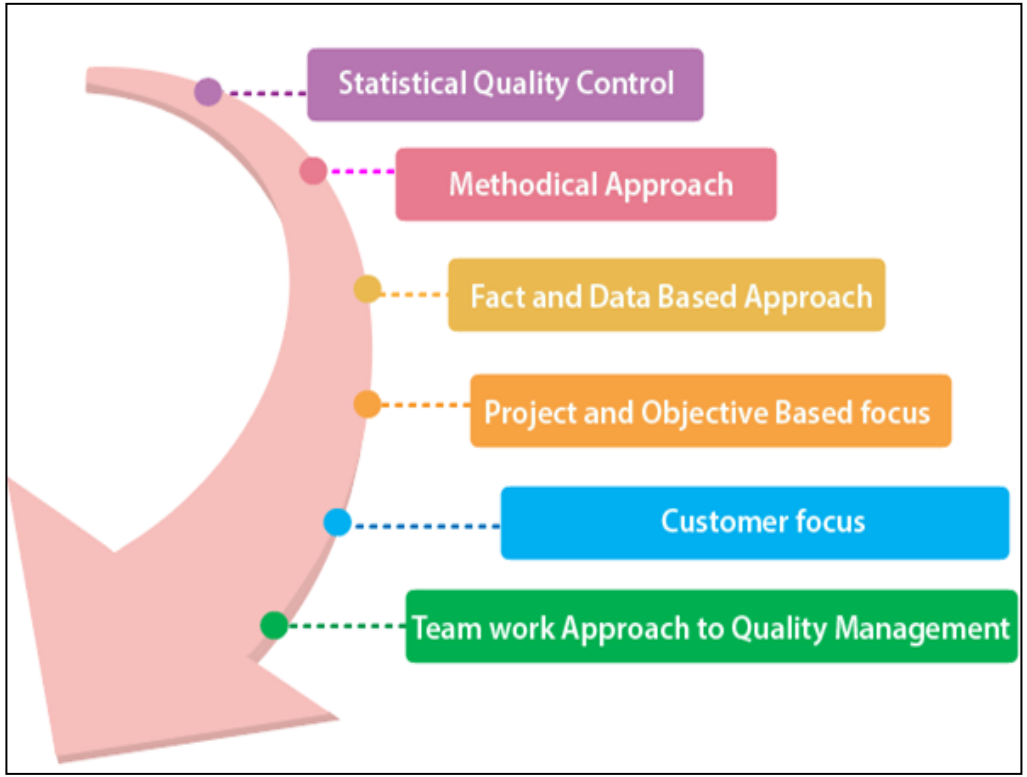
1. Statistical Quality Control

- Six Sigma is derived from the Greek Letter **σ** (Sigmat, which is used to ***denote Standard Deviation*** in statistics.
- Standard Deviation is used for ***measuring the quality of output.***

A web development team tracks the load time of a website and uses standard deviation to identify and reduce performance inconsistencies.

2. Methodical Approach

- The Six Sigma is a systematic approach of application in **DMAIC** and **DMADV** which can be used ***to improve the quality of production***.



A car manufacturing company applies DMAIC to reduce defects in engine assembly, while it uses DMADV to design a new fuel-efficient engine with high quality.

3. Fact and Data-Based Approach

- Statistical and methodical aspect of Six Sigma shows the ***scientific basis of the technique***.
- Highlights essential elements of the Six Sigma that is a fact and data-based.

A restaurant uses customer feedback data to identify the most common complaints and improves food quality based on the data.

4. Project and Objective-Based Focus

- The Six Sigma process is ***implemented to focus on the requirements and conditions.***

A software company sets a goal to reduce app crashes by 20% in the next quarter using Six Sigma.

5. Customer Focus

- The quality improvement and control standards are based on specific customer requirements.

A laptop manufacturer improves battery life based on customer demands for longer usage hours.

6. Teamwork Approach to Quality Management

- Six Sigma process requires organizations to get organized for improving quality.

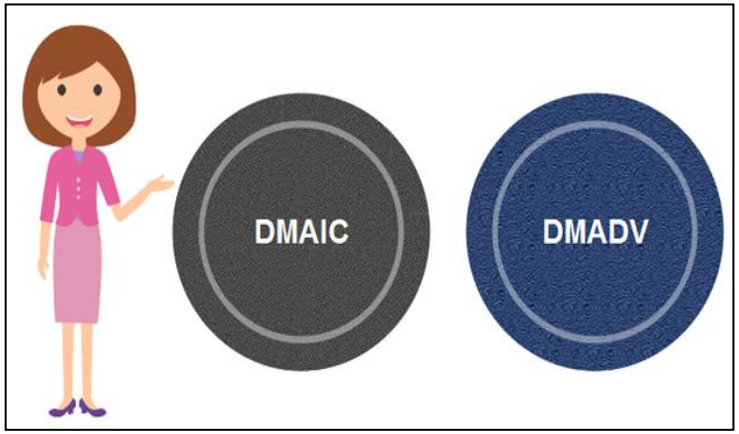
A car company forms a cross-functional team of engineers, designers, and testers to enhance vehicle safety.

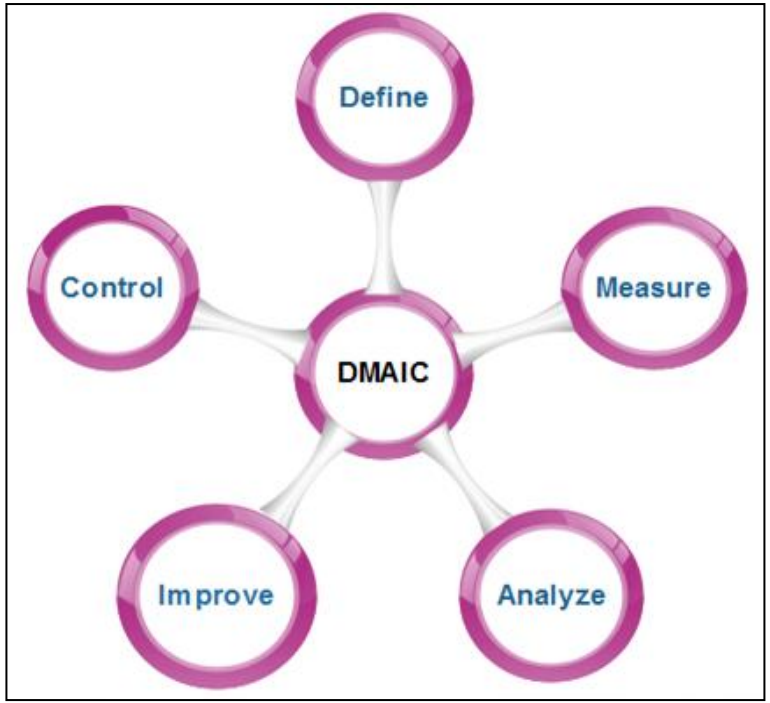
Six Sigma Methodologies

- Six Sigma projects follow two project methodologies:

1. DMAIC

- It specifies a data-driven quality strategy for ***improving processes***.
- This methodology is used to enhance an existing business process.
- **DMAIC project methodology has five phases:**
  - ***(i) Define*** : Define ***customer requirements*** and ***deliverables*** and ***project goals*** via well-defined methods of customer communication.
  - ***(ii) Measure***: Measure the ***existing process and its output*** to determine current quality performance (collect defect metrics).
  - ***(iii) Analyze***: Analyze defect metrics and determine the vital few causes.

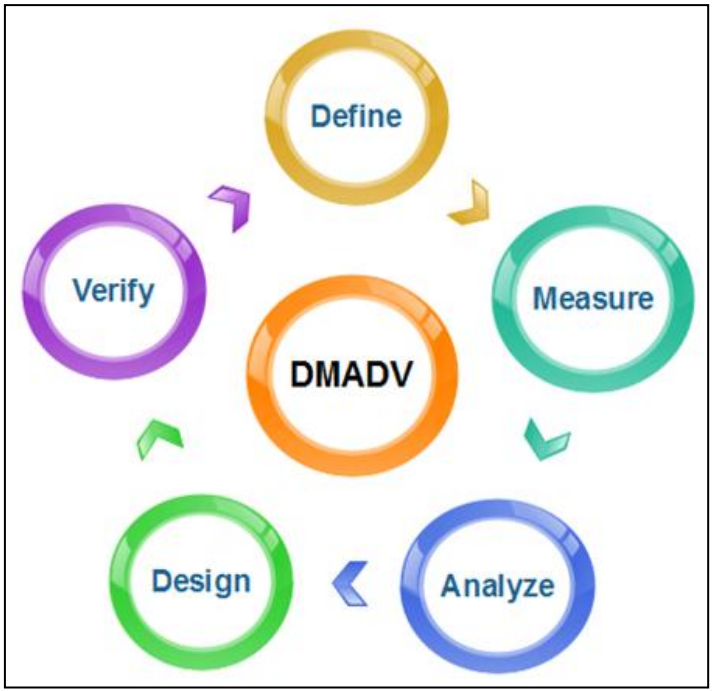




(iv) Improve**:* Improve the process by ***eliminating the root causes of defects.

***(v) Control**: Control* the process to ensure that future work does not reestablish the causes of defects.

2. DMADV



- It specifies a data-driven quality strategy for ***designing products and processes***.
- This method is used to create new product designs or process designs in such a way that it results in a more predictable, mature, and detect free performance.
  - ***(i) Define*:** It defines the ***problem*** or ***project goa***l that needs to be addressed.
  - ***(ii) Measure*:** It measures and determines the ***customer's needs and specifications***.
  - ***(iii) Analyze*:** It analyzes the process to meet customer needs.
  - ***(iv) Design*:** It can design a process that will meet customer needs.
  - ***(v) Verify*:** It can verify the design performance and ability to meet customer needs.

D

DEFINE

- Launch Team
- Establish Charter
- Plan Project
- Gather the Voice of the Consumer
- Plan for Change

M

MEASURE

- Document the Process
- Collect Baseline data
- Narrow Project Focus

A

ANALYZE

- Analyze Data
- Identify Root Cause
- Identify and Remove Wastes

I

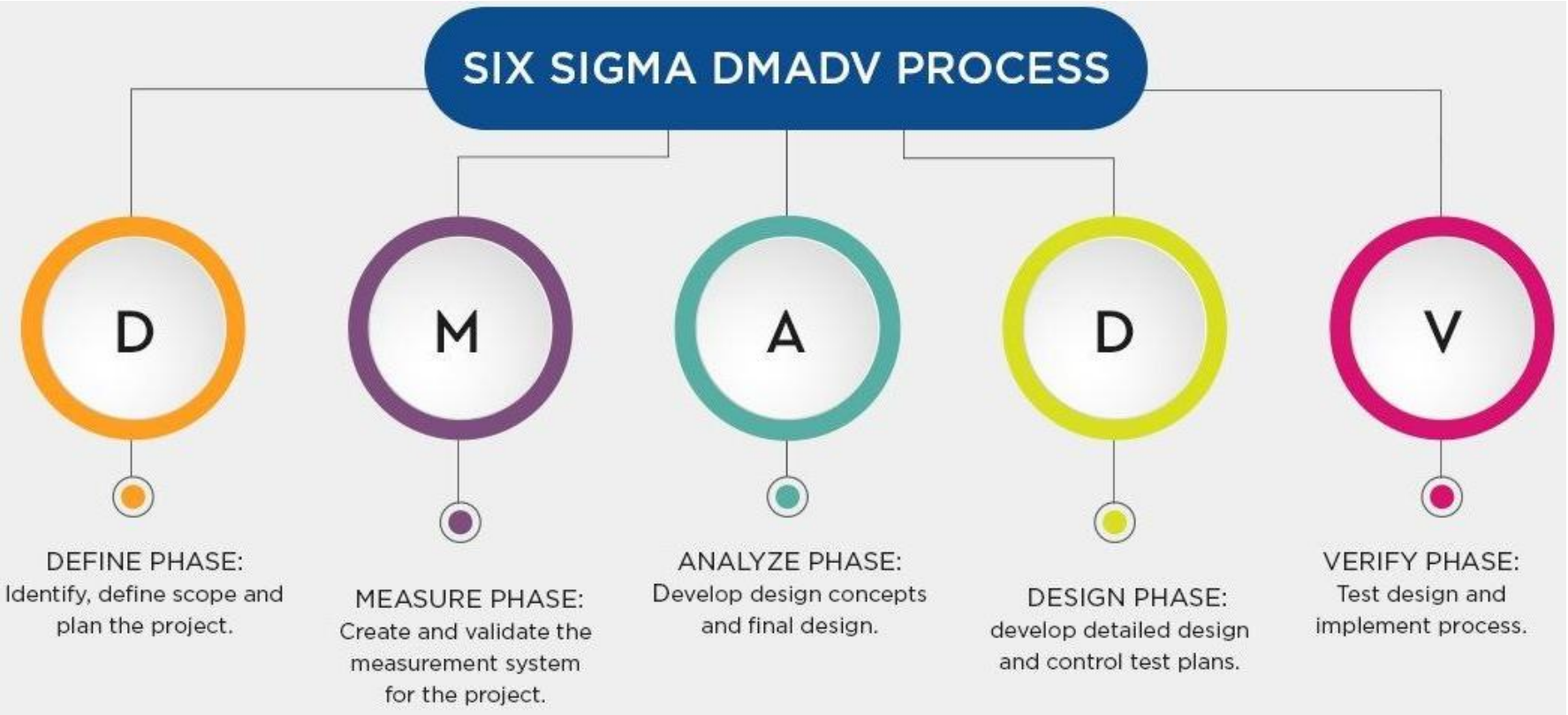
IMPROVE

- Generate Solutions
- Evaluate Solutions
- Optimize Solutions
- Pilot
- Plan and Implement

C

CONTROL

- Control de Process
- Validate Project Benefits


SIX SIGMA DMADV PROCESS
D
M
A
D
V
DEFINE PHASE:
Identify, define scope and plan the project.
MEASURE PHASE:
Create and validate the measurement system for the project.
ANALYZE PHASE:
Develop design concepts and final design.
DESIGN PHASE:
develop detailed design and control test plans.
VERIFY PHASE:
Test design and implement process.

Microsoft (MSFT) is one of the largest software producers in the world. It used Six Sigma to help eradicate defects in its systems and data centers and systematically reduce IT infrastructure failures.

The company first established standards for all of its hardware and software to create a baseline measurement for detecting defects. It then used root-cause analysis, including collecting data from past high-priority incidents, server failures, and recommendations from product group members and customers, to pinpoint potential problem areas.

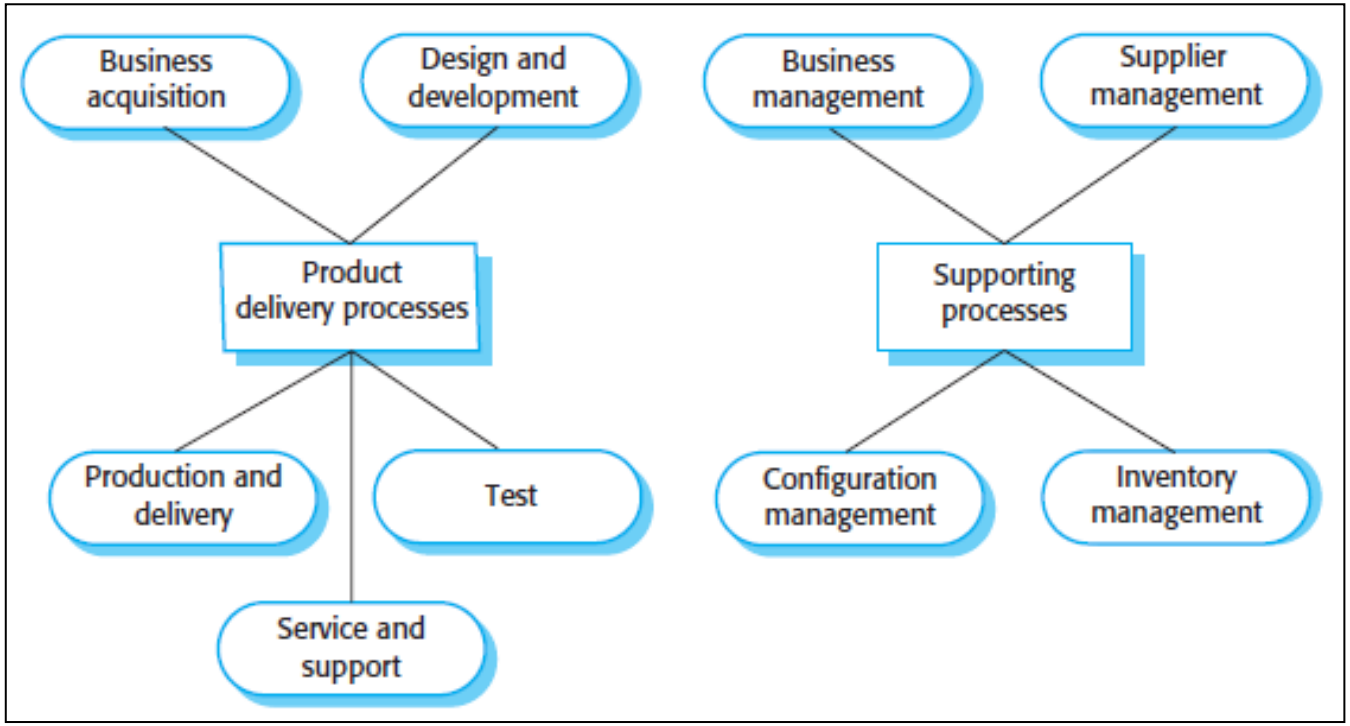
Large amounts of data were collected on a daily and weekly basis from various servers. The incidents were prioritized based on how severely the defects affected the business and the company's underlying services. Data analysis and reporting identified the specific defects, after which remediation steps for each defect were established.

As a result of Six Sigma, Microsoft says it improved the availability of its servers, boosted productivity, and increased customer satisfaction.

Quality Standards Models - ISO

- A **quality assurance system** may be defined as the ***organizational structure***, ***responsibilities***, ***procedures***, ***processes***, and ***resources*** for implementing quality management.
- The international set of standards used in the development of quality management systems in all industries is called **ISO 9000**
- **ISO 9000** standards can be applied to a range of organizations from ***manufacturing through to service industries***.
- **ISO 9001**, the most general of these standards, applies to organizations that ***design***, ***develop***, and ***maintain products***, including software.
- The ISO 9001 standard was originally developed in 1987

- The **ISO 9001** standard was significantly revised in **2000**, restructuring it around **nine core processes**.



To be ISO 9001 compliant, an organization must:

- ✓ ***Document how its processes align with the core ISO 9001 processes.***
- ✓ ***Keep records showing that the defined processes are being followed.***
- ✓ ***Include these processes and the required data collection in the company's quality manual.***

Business Acquisition: Getting new customers or projects.

Getting customers or contracts for car orders.

Design & Development: Creating and improving the product.

Designing the car and planning how it will be built.

Production & Delivery: Making and sending the product to customers.

Assembling the car and delivering it to customers.

Testing: Checking if the product works properly.

Checking if the car works well

Service & Support: Helping customers after they receive the product.

Fixing any issues and providing maintenance after customers buy the car.

Supporting Processes

Business Management: Handling company operations.

Managing finances, marketing, and operations.

Supplier Management: Working with suppliers for materials.

Getting parts and materials (engines, tires, etc.) from suppliers.

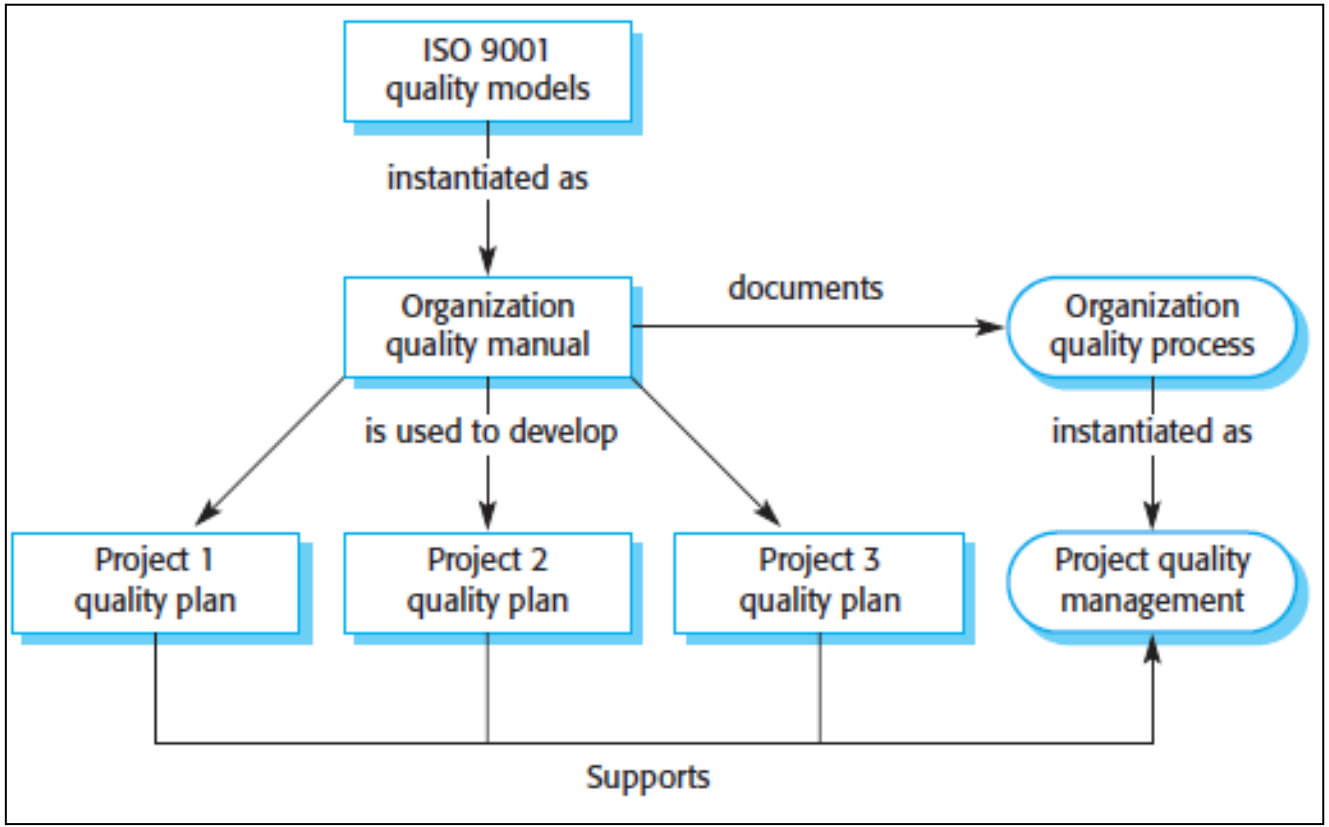
Configuration Management: Keeping track of changes and updates.

Tracking changes in car models, features, or design.

Inventory Management: Managing stock and supplies.

Keeping track of car parts and raw materials.

- The relationships between ***ISO 9001***, ***organizational quality manuals***, and ***individual project quality plans*** are shown in figure.
- This diagram explains how the general ISO 9001 standard can be used as a basis for software quality management processes.



INFO

The ISO 9001:2000 Standard

The following outline defines the basic elements of the ISO 9001:2000 standard. Comprehensive information on the standard can be obtained from the International Organization for Standardization (**www.iso.ch**) and other Internet sources (e.g., **www.praxiom.com**).

Establish the elements of a quality management system.
- Develop, implement, and improve the system.
- Define a policy that emphasizes the importance of the system.

Document the quality system.
- Describe the process.
- Produce an operational manual.
- Develop methods for controlling (updating) documents.
- Establish methods for record keeping.

Support quality control and assurance.
- Promote the importance of quality among all stakeholders.
- Focus on customer satisfaction.
- Define a quality plan that addresses objectives, responsibilities, and authority.
- Define communication mechanisms among stakeholders.

Establish review mechanisms for the quality management system.
- Identify review methods and feedback mechanisms.
- Define follow-up procedures.

Identify quality resources including personnel, training, and infrastructure elements.
- Establish control mechanisms.
- For planning
- For customer requirements
- For technical activities (e.g., analysis, design, testing)
- For project monitoring and management

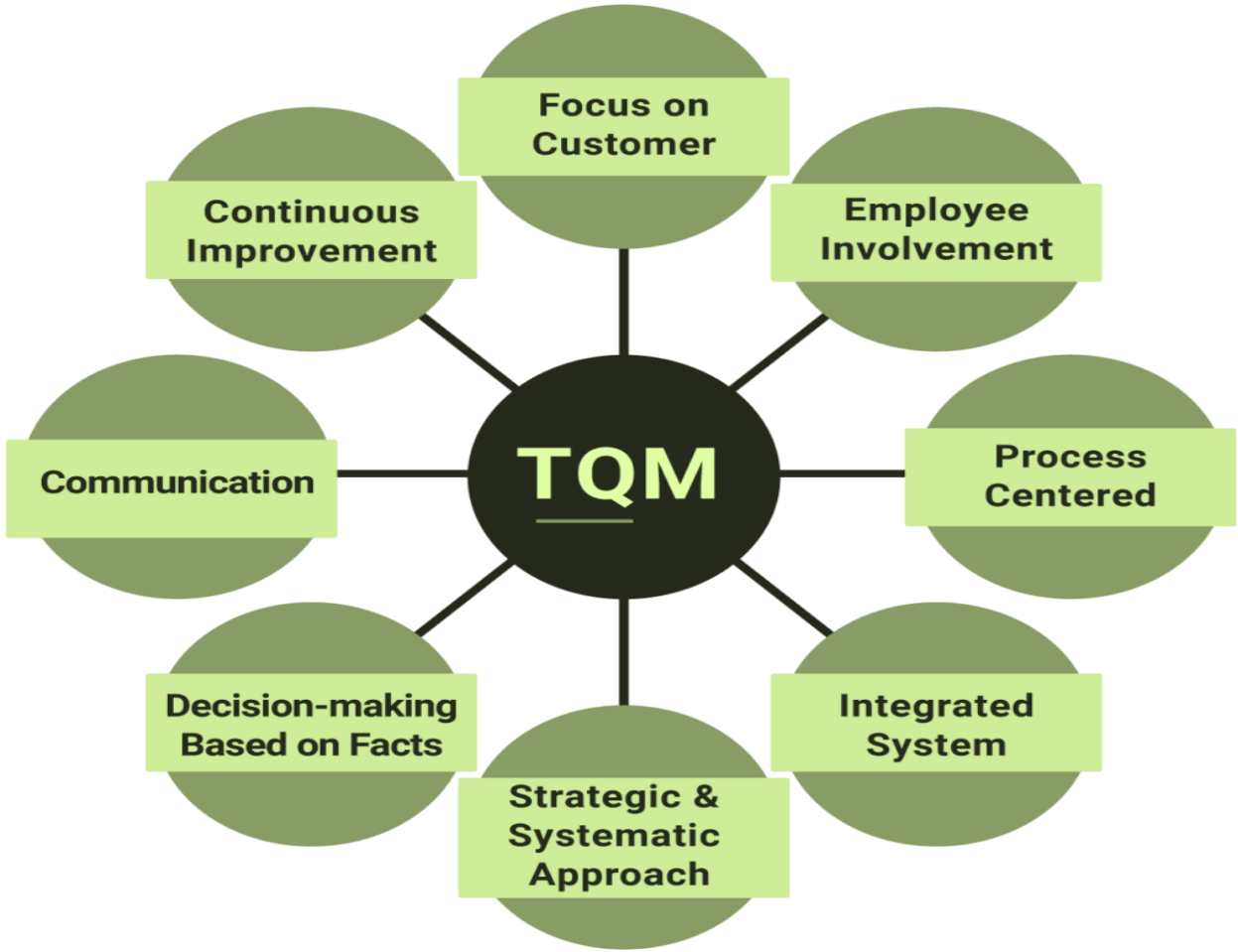
Define methods for remediation.
- Assess quality data and metrics.
- Define approach for continuous process and quality improvement.

Quality Standards Models - TQM

- Total Quality Management (TQM) is a holistic approach to quality management that focuses on continuous improvement, customer satisfaction, and the involvement of all stakeholders in an organization.
- In software engineering , TQM is applied to ensure that software development processes and products meet or exceed customer expectations by fostering a culture of quality throughout the organization.

TQM in software engineering emphasizes:

- **Customer-Centric Approach :** Ensuring that the software meets or exceeds user requirements and expectations.
- **Continuous Improvement :** Regularly refining processes and practices to enhance efficiency and quality.
- **Employee Involvement :** Engaging all team members—developers, testers, managers, and stakeholders—in achieving quality goals.
- **Process Orientation :** Standardizing and optimizing workflows to reduce errors, inefficiencies, and rework.


TQM
Focus on Customer
Employee Involvement
Process Centered
Integrated System
Strategic & Systematic Approach
Decision-making Based on Facts
Communication
Continuous Improvement

TQM - Principle

1. Customer Focus

- The primary goal of TQM is to deliver software that satisfies customer needs.
- This involves understanding user requirements, gathering feedback, and incorporating it into the development process.
- Example: Regularly conducting user surveys or usability testing to ensure the software aligns with user expectations.

2. Continuous Improvement

- TQM promotes a mindset of constant refinement.
- Teams analyze processes, identify bottlenecks, and implement improvements iteratively.
- Example: Using Agile methodologies to incorporate feedback and improve features in each sprint.

3. Employee Empowerment

- TQM encourages active participation from all team members Developers, testers, and other stakeholders are empowered to suggest improvements and take ownership of quality.
- Example: Conducting brainstorming sessions where team members propose ideas to reduce defects or streamline workflows.

4. Process-Centric Approach

- TQM emphasizes defining, documenting, and standardizing processes to ensure consistency and repeatability insoftware development.
- Example: Implementing coding standards, automated testing frameworks, and code review processes to maintain high-quality outputs.

5. Data-Driven Decision Making

- Decisions in TQM are based on measurable data rather than assumptions.
- Metrics like defect rates, cycle times, and user feedback are analyzed to guide improvements.
- Example: Using tools like JIRA or SonarQube to track bugs and technical debt, then using this data to prioritize fixes.

6. Leadership Commitment

- Leadership plays a critical role in fostering a culture of quality. Managers must support TQM initiatives by providing resources, setting clear goals, and leading by example.
- Example: Senior management conducts regular reviews of quality metrics and allocates budgets for training and tools.

7. Systematic Problem Solving

- TQM uses structured methods like PDCA (Plan-Do-Check-Act) or Root Cause Analysis (RCA) to address issues systematically.
- Example: Applying RCA to identify why a specific bug occurred and implementing preventive measures to avoid recurrence.

8. Long-Term Thinking

- TQM focuses on sustainable improvements rather than short-term fixes. It aims to build robust processes that consistently deliver high- quality software over time.
- Example: Investing in automation tools for testing and deployment to improve long-term efficiency.

TQM - Implementation

1. Requirement Gathering and Analysis	• Work closely with customers to understand their needs and expectations. • Use techniques like prototyping or user stories to clarify requirements and ensure alignment.
2. Design and Development	• Follow standardized design principles and coding guidelines. • Use tools like version control systems (e.g., Git) and code review platforms to maintain consistency.
3. Testing and Quality Assurance	• Implement rigorous testing processes, including unit testing, integration testing, and user acceptance testing (UAT). • Use automated testing tools to ensure comprehensive coverage and faster feedback loops.
4. Deployment and Maintenance	• Monitor software performance in production using tools like application performance monitoring (APM). • Continuously gather user feedback and release updates to address issues or add new features.
5. Feedback Loop	• Collect feedback from users, stakeholders, and team members after each release. • Use this feedback to refine processes and improve future releases.

TQM - Challenges

a. Cultural Resistance

- Employees may resist changes to established workflows or struggle to adapt to a quality- focused mindset.

b. Resource Constraints

- Implementing TQM requires investment in tools, training, and process redesign, which may strain budgets.

c. Measurement Complexity

- Defining and tracking meaningful quality metrics can be challenging, especially in complex software projects.

d. Time-Consuming

- TQM is a long-term strategy, and results may not be immediately visible, which can test organizational patience.

TQM - Example

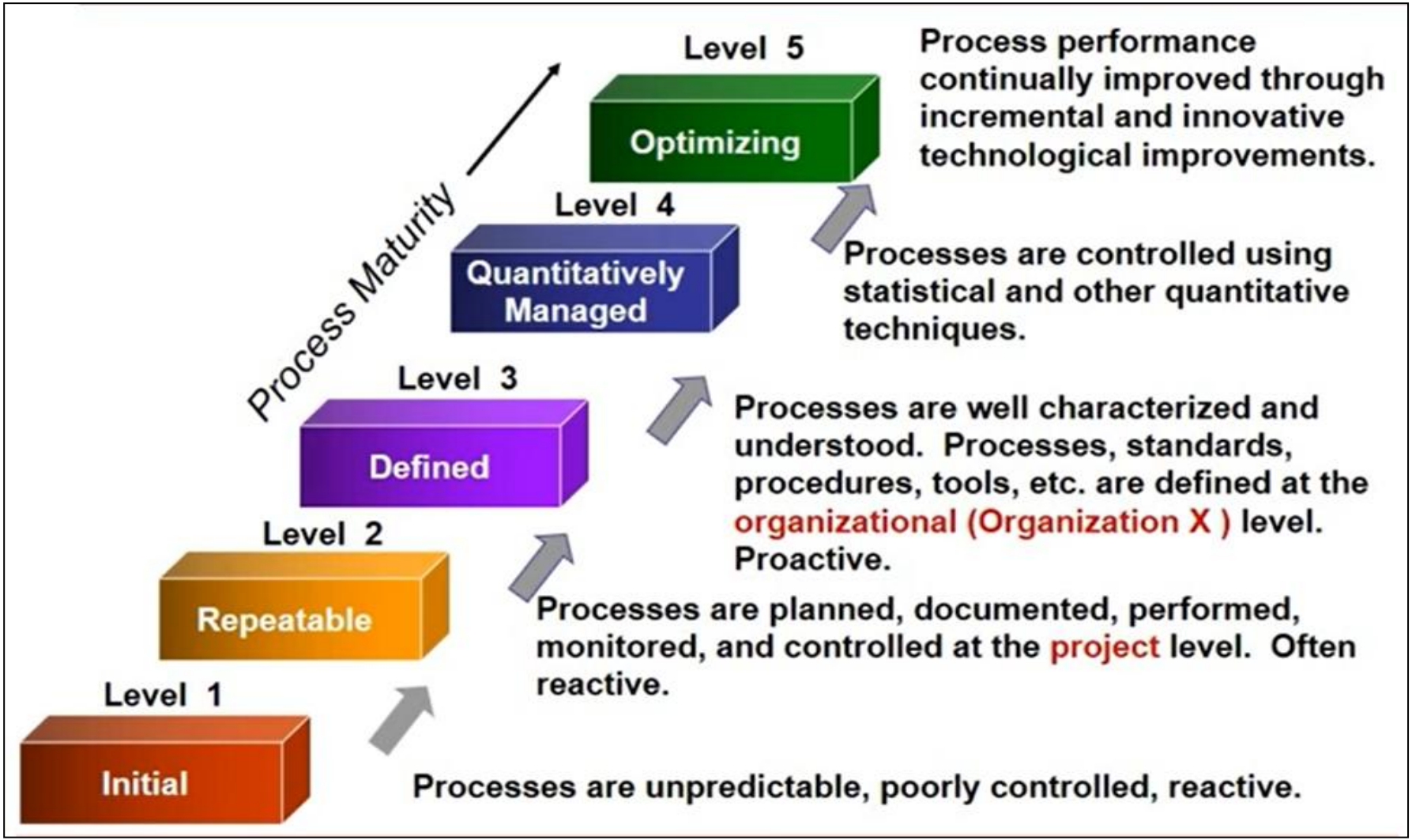
- A software company adopts TQM to improve its product quality:
- **Customer Focus** : Conducts user interviews to identify pain points in the current software.
- **Continuous Improvement**: Implements Agile practices to release incremental updates based on feedback.
- **Employee Involvement** : Holds weekly meetings where developers and testers discuss ways to reduce bugs.
- **Process Standardization** : Introduces coding standards and mandatory peer reviews to catch issues early.
- **Data-Driven Decisions**: Tracks defect rates and user satisfaction scores to measure progress.
- **Outcome** : Over time, the company reduces post-release defects by 40% and improves customer retention.

Process Improvement Models: CMM & CMMI

CMM

- The Capability Maturity Model (CMM) is a procedure used to ***develop and refine an organization's software development process***.
- It also provides guidelines to enhance further the maturity of the process used to develop those software products.
- It is based on intense feedback and development practices adopted by the most successful organizations worldwide.
- CMM was developed and is promoted by the **Software Engineering Institute** (SEI), a research and development center promote by the **U.S. Department of Defense** (DOD).
- **Importance of Capability Maturity Model**
  - ***Optimization of Resources***
  - ***Comparing and Evaluating***
  - ***Management of Quality***
  - ***Enhancement of Process***
  - ***Increased Output***

- There are five levels to the CMM development process.



Five levels to the CMM development process

Level	Name	Meaning	Example
1	**Initial**	Unpredictable, no clear process.	The bakery bakes different items each day with no plan.
2	**Repeatable**	Some process exists, can be repeated.	The bakery uses a recipe book for common items like bread.
3	**Defined**	All processes documented and standardized.	The bakery follows fixed recipes, schedules, and roles.
4	**Quantitatively Managed**	Uses data and statistics to improve.	The bakery tracks how many items sell daily and adjusts baking.
5	**Optimizing**	Continuously improving using feedback and innovation.	The bakery experiments with new recipes and customer feedback.

CMMI (Capability Maturity Model Integration)

- CMMI is a guide that helps organizations improve the way they work so they can do things better, faster, and with higher quality.
- The Capability Maturity Model Integration (CMMI) is based on four pillars:

Capability :

- A capable process is one that ***works consistently*** and gives results within expected limits.
- You can trust it to do the same good job again and again.

Maturity

- Maturity in a company means:

✓ **Clear processes** – Everything the company does is well-documented.

They follow a well-documented process for writing, reviewing, and testing code.

✓ **Clear roles** – Everyone knows what is expected of them.

Developers know who handles coding, who reviews code, and who tests it.

✓ **Consistency** – Success doesn't rely on a few "heroes" doing extraordinary work.

If one developer is out, another can easily step in using the same process and tools.

✓ **Smart decisions** – Choices are based on proper analysis, not guesswork.

Before adding a new feature, they look at user data and discuss the pros and cons as a team.

Model

- A model is a basic structure or guide for how to do something.
- **CMMI** is a model that ***helps companies set up good processes***.
- It doesn't tell you exactly *how* to do things — it just shows **what** needs to be in place.
- The goal is to help you build **efficient and effective** ways of working.

Integration

- CMMI was ***created by combining the best practices from several earlier models*** used to improve software and system development.
- It brought together what worked well in different approaches into one unified model.
- The CMMI represents a **process meta-model** in two different ways:
  (1) as a "continuous" model (Improve one process area at a time)
  (2) as a "staged" model (Improve the organization level-by-level)

- The CMMI defines each **process area** in terms of **"specific goals"** and the **"specific practices"** required to achieve these goals.
- **Specific goals** ***establish the characteristics*** that must exist if the activities implied by a process area are to be effective.
- **Specific practices** ***refine a goal into a set of process-related activities***.

Imagine you're planning a birthday party.
SG:
Organize a successful birthday party.
SPs:
SP1: Choose a venue
SP2: Send invitations
SP3: Order food and cake
SP4: Plan fun activities

- Each **process area** is formally assessed against specific goals and practices and is rated according to the following capability levels:

(i) Level 0: *Incomplete*

- The process is ***missing or not done*** properly..

You try to bake a cake without a recipe. You forget things, and the cake doesn't turn out well.

(ii) Level 1: *Performed*

- The basic goals for the process are met.
- The required tasks are being done to create the expected results.

You use a recipe and bake the cake. It works, but you don't check or follow any extra rules.

(iii) Level 2: *Managed*

- Level 1 goals are achieved.
- The work follows company rules.
- People have the tools and support they need.
- Stakeholders are involved when needed.
- Work is tracked, checked, and follows the planned process.

You follow the recipe and kitchen rules. You have everything you need, and someone makes sure you're doing it right.

Level 3: ***Defined***

- The process follows the company's standard way of doing things.
- It's customized and includes helpful tools and tips to do better work.
- Professional and consistent every time

You follow a bakery's official recipe, use special tools, and follow best practices.

Level 4: ***Quantitatively managed***

- You measure performance and use data.

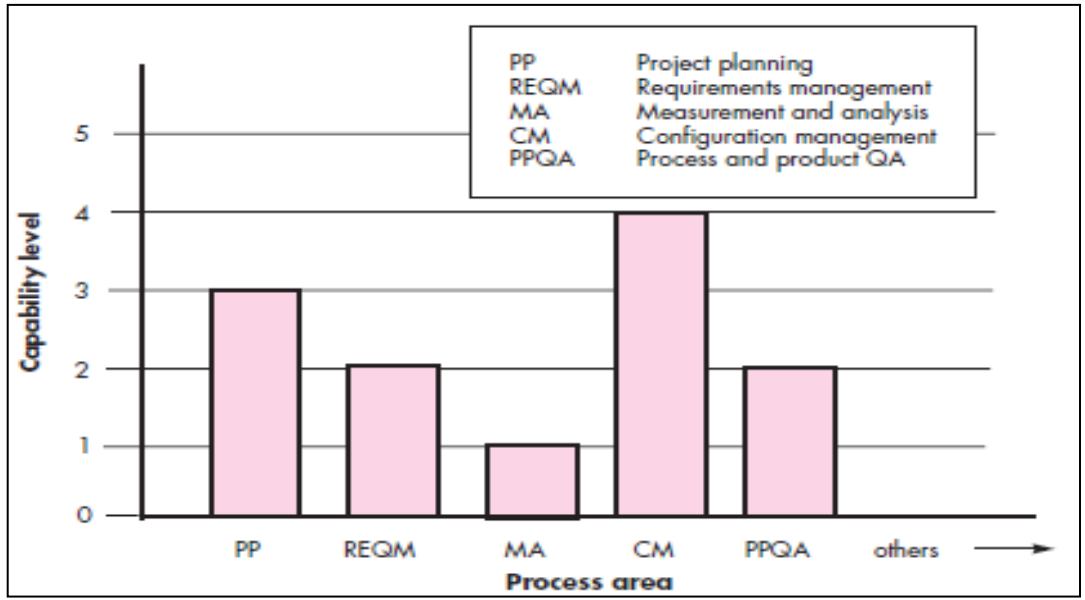
You measure how long to bake, exact temperature, and adjust based on past cake results.

Level 5: ***Optimized***

- You keep improving using data and feedback..
- Changes are made to stay up to date and make things even better over time.

You try new ingredients, adjust the recipe, and improve based on customer reviews.

- The continuous CMMI meta model describes a process in two dimensions ie **Process area** and **Capability area**.



Process Area	Capability Level	Meaning
PP	Level 3	Project planning is well defined and standardized.
REQM	Level 2	Requirements are managed but not yet standardized.
MA	Level 1	Basic measurement is happening, but not consistently.
CM	Level 4	Configuration is controlled using data and measurements.
PPQA	Level 2	Quality checks are being done and managed.

- **Project planning** is one of eight process areas defined by the CMMI for "project management" category.
- The **Specific Goals (SG)** and the associated **Specific Practices (SP)** defined for **project planning** are

SG 1 Establish Estimates

SP 1.1-1: Estimate the Scope of the Project

Build a web/mobile app to let patients' book, reschedule, and cancel appointments with doctors.

SP 1.2-1: Establish Estimates of Work Product and Task Attributes

Estimate user interface screens (10), database tables (8), backend APIs (15), testing cases (30)

SP 1.3-1: Define Project Life Cycle

Use Agile methodology with 4 sprints, each lasting 2 weeks.

SP 1.4-1: Determine Estimates of Effort and Cost

Estimate total 400 hours of effort and $25,000 cost over 2 months.

SG 2 Develop a Project Plan

SP 2.1-1: Establish the Budget and Schedule

Budget approved at $25,000; timeline: April 1 - June 30.

SP 2.2-1: Identify Project Risks

Delays in API integration, scope creep, data privacy compliance

SP 2.3-1: Plan for Data Management

Use PostgreSQL; ensure daily backups and secure patient data storage

SP 2.4-1: Plan for Project Resources

1 PM, 2 Developers, 1 Tester, 1 UI Designer.

SP 2.5-1: Plan for Needed Knowledge and Skills

Team trained in HIPAA compliance and React development.

SP 2.6-1: Plan Stakeholder Involvement

Weekly demo and feedback from hospital admin

SP 2.7-1: Establish the Project Plan

Full plan documented in project management tool

SG 3 Obtain Commitment to the Plan

SP 3.1-1: Review Plans That Affect the Project

Ensure alignment with IT infrastructure and marketing launch plans.

SP 3.2-1: Reconcile Work and Resource Levels

Adjust scope to fit available developer capacity.

SP 3.3-1: Obtain Plan Commitment

All team members and stakeholders sign off on the plan.

- The CMMI also defines a set of five **Generic Goals(GG)** and related practices for each process area.
- Each of the five generic goals corresponds to one of the five capability levels

GG 1 Achieve Specific Goals

GP 1.1 Perform Base Practices

GG 2 Institutionalize a Managed Process

GP 2.1 Establish an Organizational Policy
GP 2.2 Plan the Process
GP 2.3 Provide Resources
GP 2.4 Assign Responsibility
GP 2.5 Train People
GP 2.6 Manage Configurations
GP 2.7 Identify and Involve Relevant Stakeholders
GP 2.8 Monitor and Control the Process
GP 2.9 Objectively Evaluate Adherence
GP 2.10 Review Status with Higher- Level Management

GG 3 Institutionalize a Defined Process

GP 3.1 Establish a Defined Process
GP 3.2 Collect Improvement Information

GG 4 Institutionalize a Quantitatively Managed Process

GP 4.1 Establish Quantitative Objectives for the Process
GP 4.2 Stabilize Subprocess Performance

GG 5 Institutionalize an Optimizing Process

GP 5.1 Ensure Continuous Process Improvement
GP 5.2 Correct Root Causes of Problems

- The staged CMMI model defines the same process areas, goals, and practices as the continuous model.
- The primary difference is that the staged model defines five maturity levels, rather than five capability levels.
- To achieve a maturity level, the specific goals and practices associated with a set of process areas must be achieved.
- The relationship between maturity levels and process areas is shown

Level	Focus	Process Areas
Optimizing	*Continuous process improvement*	Organizational innovation and deployment Causal analysis and resolution
Quantitatively managed	*Quantitative management*	Organizational process performance Quantitative project management
Defined	*Process standardization*	Requirements development Technical solution Product integration Verification Validation Organizational process focus Organizational process definition Organizational training Integrated project management Integrated supplier management Risk management Decision analysis and resolution Organizational environment for integration Integrated teaming
Managed	*Basic project management*	Requirements management Project planning Project monitoring and control Supplier agreement management Measurement and analysis Process and product quality assurance Configuration management
Performed		

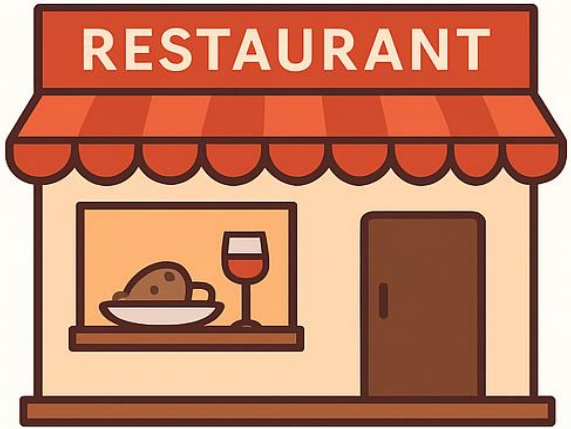
CMM

- **Focus:** Software processes
- **Scope:** Narrow
- **Approach:** Staged only

Exemple: Cake baking

CMMI

- **Focus:** Software + Systems + Services + Management
- **Scope:** Broad / integrated
- **Approach:** Staged and continuous



Quality Control and Quality Assurance

Quality Assurance

- QA is a ***proactive***, ***process-oriented*** approach that focuses on **preventing defects** in the process used to make a product or deliver a service.
- Key Characteristics
  - ➢Ensures process quality
  - ➢Prevents issues before they occur
  - ➢Involves standards, procedures, and guidelines
  - ➢Focuses on continuous improvement
  - ➢Typically includes audits, training, process checklists, and reviews
- QA includes
  - ➢writing coding standards
  - ➢conducting code reviews
  - ➢defining testing protocols. Etc.

Quality Control (QC)

- QC is a ***reactive***, ***product-oriented*** approach that focuses on ***identifying defects*** in the final product.
- Key Characteristics
  - ➢ Ensures product quality
  - ➢ Detects issues after they occur
  - ➢ Involves testing, inspection, and validation
  - ➢ Focuses on meeting quality specifications
  - ➢ Usually involves manual/automated testing.
- QC would involve
  - ✓ Executing test cases
  - ✓ Finding bugs
  - ✓ Reporting issues before a release.

Quality Management

- The process of ensuring that products or services meet consistent standards and satisfy customer expectations.
- In simple terms, it's about making things right, keeping them right, and improving them over time.

Quality Management = Plan it right (Planning) + Do it right (QA) + Check it right (QC) + Keep getting better (Improvement)

Quality Management – Four Parts

Quality Planning

- Deciding **what "quality" means** for the project/product.
- Setting **goals, standards, and procedures** before work begins.
- Example: Defining performance benchmarks, security standards, or coding guidelines in a software project.

Quality Assurance

- **Creating and following processes** to avoid mistakes.
- Focuses on **how the work is done**.
- Example: Having code reviews or CI/CD pipelines to maintain code quality.

Quality Control

- **Checking the final product** to find and fix defects.
- Focuses on **what is produced**.
- Example: Testing a software app to find bugs before release.

Continuous Improvement

- **Learning from mistakes** and improving processes.
- Common models: **PDCA (Plan-Do-Check-Act)**, **Six Sigma**, **Kaizen**.
- Example: Collecting feedback from users after product launch and using it to improve the next version.

Quality Management – Key Methods

- PDCA Cycle (Plan–Do–Check–Act)
- Six Sigma
- Statistical Process Control (SPC)
- Total Quality Management (TQM)
- ISO 9001 (Quality Standards)
- Failure Mode and Effects Analysis (FMEA)

Method	Focus	Key Benefit
TQM	Whole organization	Everyone improves quality
PDCA	Iterative improvement	Simple and flexible
Six Sigma	Reducing defects	Data-driven accuracy
SPC	Monitoring processes	Early detection of issues
FMEA	Risk analysis	Prevents major failures
ISO 9001	Compliance	Trusted industry standard
Kaizen	Small improvements	Continuous team learning

Quality Factors

- Quality Factors refer to the key characteristics that define how good a software product is.
- These are used to evaluate, measure, and ensure software quality from both a technical and user perspective

Quality Factor	What it Means
Functionality	Meets user needs correctly
Performance	Fast and resource-efficient
Reliability	Works consistently without crashing
Usability	Easy to use and understand
Maintainability	Easy to fix and update
Portability	Can run on different systems
Security	Safe from threats and data leaks
Testability	Easy to test for bugs and errors

1. Functionality

- Does the software do what it's supposed to do?
  - **Correctness**: Produces the right results

Online Banking App allows users to check their balance, make transfers, and view transaction history.

  - **Security**: Protects against unauthorized access

The app requires a secure login using a username and password, with additional 2-factor authentication (2FA).

  - **Interoperability**: Works well with other systems

The app integrates with other systems, like credit card processing or external payment gateways.

2. Performance Efficiency

- Performance efficiency is about how well the software uses system resources (like CPU, memory, and bandwidth) to perform its tasks.
  - **Response Time**

When you tap on "Check Balance", it shows your balance in 2 seconds.

  - **Throughput**

App can handle 1000 balance requests per second from users across the country

  - **Resource Usage**

App uses low CPU and memory, so it doesn't drain your battery quickly

3. Reliability

- Can the software perform well under different conditions?
  - **Fault Tolerance**: Works even when some parts fail

If the server in one region crashes, the app switches to another server automatically and messages still go through.

  - **Availability**: Ready when needed

You open the app at 2 AM or 2 PM—it works. It's available 24/7.

  - **Recoverability**: Recovers from crashes

If your internet cuts out while sending a message, the app stores it and sends it automatically once you're back online.

4. Usability

- Refers to how easy and efficient it is for users to learn and use a software application.
  - **Learnability**

The app shows a quick tutorial highlighting key buttons like "Search," "Cart," and "Place Order."

  - **Operability**

Adding and removing items from the cart is simple.

  - **User Error Protection**

If the user forgets to select a delivery address, the app shows a clear message: "Please choose a delivery location."

5. Maintainability

- Refers to how easily a software system can be modified to fix defects, update features, or enhance functionality.
  - **Modularity**

Each operation (add, subtract, etc.) is in a separate function.

  - **Reusability**

Functions like add() can be reused in other programs without changes.

  - **Analyzability**

Clear function names and a simple main block make it easy to understand.

6. Portability

- Portability means your app can work across platforms easily
  - **Adaptability**

The app adjusts its layout when run on Windows, macOS, and Linux without crashing or needing code changes.

  - **Installability**

You can easily install the app on different operating systems using simple installers or packages

  - **Replaceability**

You uninstall your old note-taking app and replace it with this new one without any issues or system conflicts.

7. Security

- Protects data and resists attacks
  - **Confidentiality**

Your password and bank balance are only visible to you.

  - **Integrity**

You transfer ₹1000 to a friend. The app ensures the amount is not changed to ₹10,000 by mistake or by an attacker.

  - **Authentication**

When you log in, the app checks your username and password.

Quality Factors by McCalls

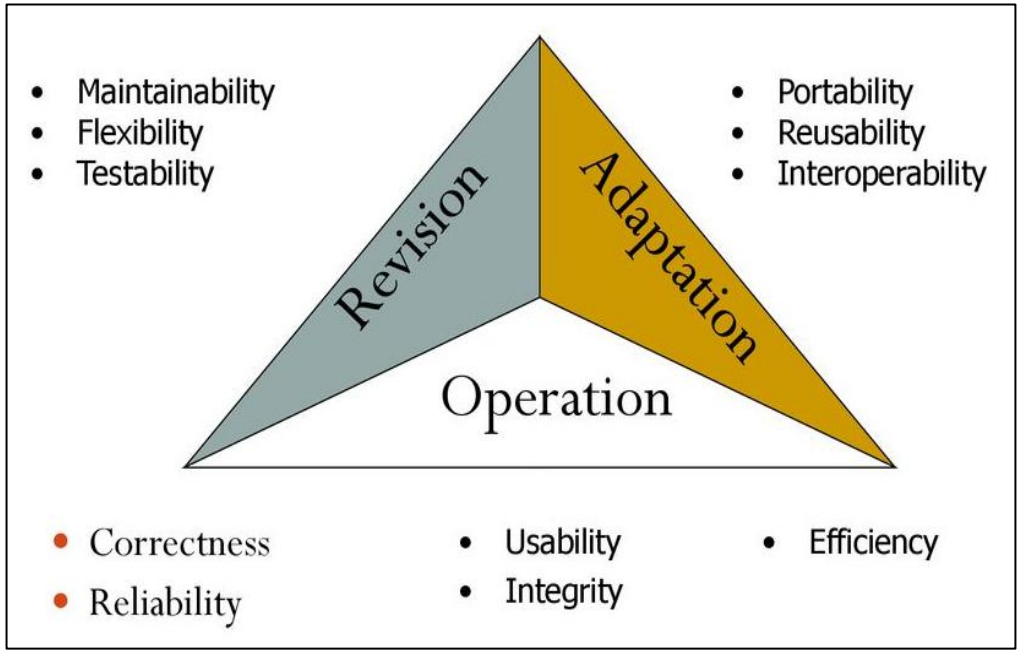
- McCall's software quality model categorizes quality factors into three main areas:

Product Revision

- **Maintainability** - *Can I fix it?*
- **Flexibility** - *Can I change it?*
- **Testability** - *Can I test it?*

Product Transition

- **Portability** - *Will I be able to use on another machine?*
- **Reusability** - *Will I be able to reuse some of the software?*
- **Interoperability** - *Will I be able to interface it with another application?*

Product Operation

- **Correctness** - *Does it do what I want?*
- **Reliability** - *Does it do it accurately all the time?*
- **Efficiency** - *Will it run on my machine as well as it can?*
- **Integrity** - *Is it secure?*
- **Usability** - *Can I run it?*